PORTFOLIO PROJECT 02 · DATABASE SCHEMA GOVERNANCE

schema-drift-checker

8/8 + PG16
tests passing

Compares the schema the application code expects with what is really deployed in Oracle or PostgreSQL – then grades every difference and drafts the fix.

Python 3.10+ · openpyxl · Oracle · PostgreSQL · Hibernate DDL · SQL / PL/SQL · pytest · GitHub Actions

Sallehuddin Mohd Zakinuddin · Senior Application Developer · Portfolio project on GitHub

1 Problem Statement

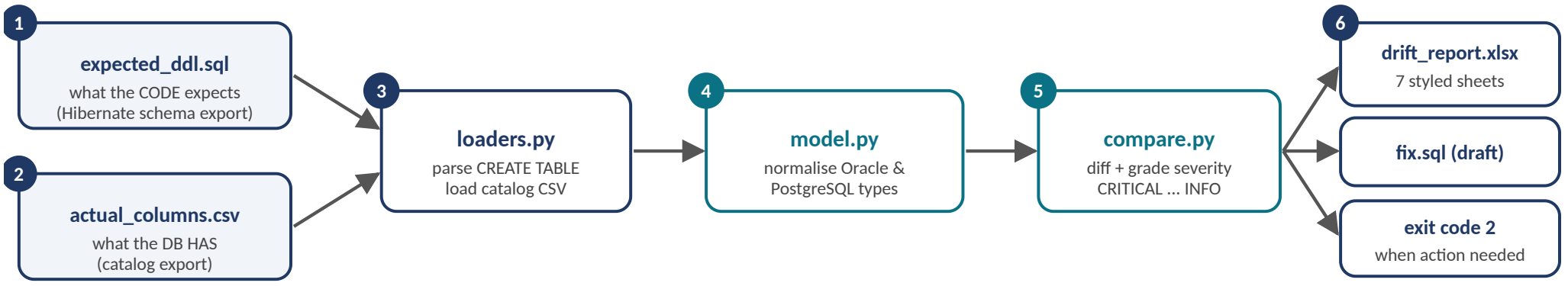
One application, many environments (states, tenants, dev → prod): the schemas **slowly drift apart** – a hotfix ALTER run in one place only, a column widened in code but not in the DB, forgotten backup tables. Nobody notices until production breaks:

- `value too long for type varchar(100)` – DB column narrower than the entity
- `ORA-00001: unique constraint violated` – sequence fell behind `MAX(id)`
- An entity field mapped to a column that does not exist

2 Objectives

1. Compare what the **code expects** with what is **actually deployed**, column by column.
2. Work **across vendors**: Oracle-style Hibernate DDL vs. a PostgreSQL DB.
3. **Grade** every difference so the team fixes what breaks production first.
4. Produce a report managers can read (Excel) and a **draft fix script** DBAs can review.
5. Detect **sequence drift** and generate a safe resync script.

3 System Architecture



① ② **Two inputs.** Expected DDL from the codebase; live column list exported with the queries in `sql/`.

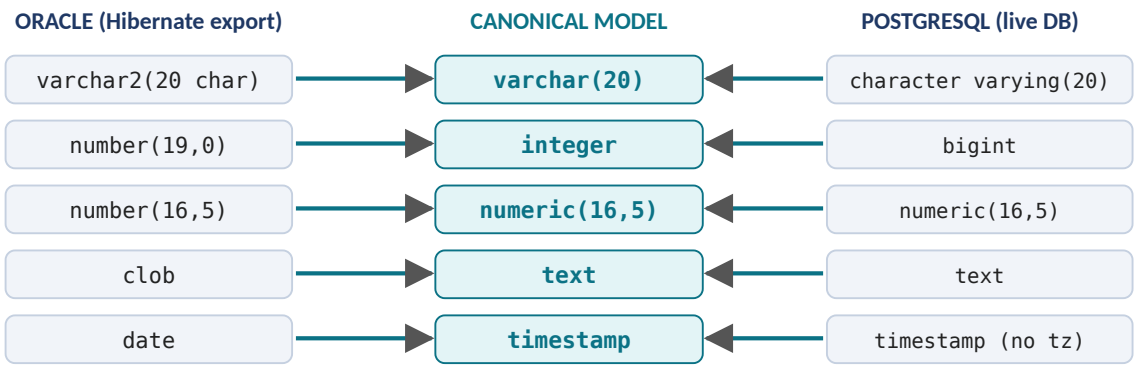
③ **Load.** Parse `CREATE TABLE` (skipping constraints) and the catalog CSV into one Table/Column model.

④ **Normalise.** Map vendor type names to one family so `varchar2` and `character varying` compare equal.

⑤ **Compare.** Missing/extra tables and columns, junk tables, then grade each type mismatch.

⑥ **Output.** Excel report, draft ALTER script, exit code for scheduled checks.

4 Key Idea – Type Normalisation



Why it matters: without this step every single column would be reported as different, because Oracle and PostgreSQL spell the same type differently. Note `number(19,0)` is really an integer – Hibernate maps Java `Long` to it.

5 Severity Grading

Level	Meaning
CRITICAL	Incompatible types (`varchar` vs `numeric`)
HIGH	DB column **narrower** – inserts fail / truncate
MEDIUM	Nullability differs, or compatible family
LOW	DB column wider than expected
INFO	Same after normalisation

Sample run – mismatches by severity:



6 Results on the sample schema

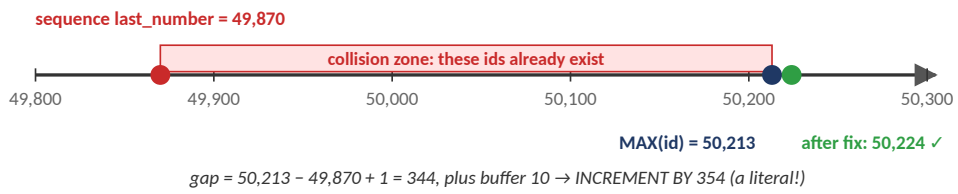
4	1	2	3
tables compared	table missing in DB	junk / backup tables	HIGH risk columns

7 Excel sheets: KPI summary, missing/extra/junk tables, missing/extra columns, colour-coded type mismatches.

```
-- draft fix.sql (REVIEW BEFORE RUNNING)
ALTER TABLE payment ADD COLUMN receipt_no varchar(30);
ALTER TABLE payment ALTER COLUMN payer_name TYPE varchar(250);
```

8 unit tests · CI also runs it end to end on a real PostgreSQL 16 database.

7 Sequence Drift – the ORA-00001 story



gap = 50,213 − 49,870 + 1 = 344, plus buffer 10 → INCREMENT BY 354 (a literal!)

Fix pattern (Oracle): `ALTER SEQUENCE s INCREMENT BY 354;` → `SELECT s.NEXTVAL FROM dual;` → `ALTER SEQUENCE s INCREMENT BY 1;` The tool computes the gap for you, because Oracle will not evaluate `50213 - 49870` inside DDL. PostgreSQL is simpler: `setval('s', MAX(id))`.

8 Key Concepts Learned

- **Source of truth.** The code (entities) defines the contract; the database must follow it.
- **Drift compounds** down an inheritance chain of environments.
- **Rank, don't dump.** A graded list gets fixed; a raw list of 500 does not.

9 Limitations & Future Work

- Parses exporter-style DDL, not every vendor extension.
- Next: read the live DB directly instead of CSV.
- Next: compare indexes, FKs and master data.

10 Interview Talking Points

Q: Why is a narrower DB column HIGH but a wider one LOW?
Narrower breaks inserts today; wider only wastes space.

Q: Why generate a draft, not run it?
Altering a big table can lock it – a human picks the time.

Portfolio project · fictional data · MIT licence